GSB - DOCUMENTATION TECHNIQUE

Application de suivi des comptes-rendus de visite médicale

Architecture MVC — PHP / MySQL

Avril 2026

I. Présentation du contexte GSB

Galaxy Swiss Bourdin (GSB) est un laboratoire pharmaceutique issu de la fusion entre le géant américain Galaxy et le conglomérat européen Swiss Bourdin.

L'entreprise compte 480 visiteurs médicaux en France métropolitaine (Corse comprise), et 60 dans les départements et territoires d'outre-mer. Les territoires sont répartis en 6 secteurs géographiques (Paris-Centre, Sud, Nord, Ouest, Est, DTOM Caraïbes-Amériques, DTOM Asie-Afrique).

Les *visiteurs médicaux* (ou *délégués*) démarchent les médecins, pharmaciens, infirmières et autres métiers de santé susceptibles de prescrire aux patients les produits du laboratoire.

L'objectif d'une visite est d'actualiser et rafraîchir la connaissance des professionnels de santé sur les produits de l'entreprise. Les visiteurs ne font pas de vente, mais leurs interventions ont un impact certain sur la prescription de la pharmacopée du laboratoire.

Pour donner une organisation commune aux délégués médicaux, l'entreprise a adopté l'organisation de la flotte de visiteurs existant chez Galaxy, selon un système hiérarchique par région et, à un niveau supérieur, par secteur géographique (Sud, Nord, Paris-Centre, Antilles-Guyane, etc).

II. Présentation du projet

- Étude de l'existant

Les visiteurs médicaux doivent réaliser un compte rendu de chaque visite effectuée.
Depuis 2003, une application ACCESS a été développée et mise à disposition des visiteurs médicaux afin de faire la saisie des comptes rendus et d'assurer un suivi de leurs visites.

- Critique de l'existant

Suite à la réorganisation de l'entreprise, cette application est devenue obsolète. Elle ne permet pas de gérer les nouveaux besoins métier, ne supporte pas les terminaux mobiles et ne garantit pas un niveau de sécurité suffisant pour une application intranet.

- Expression des besoins

L'entreprise souhaite :

- considérer l'ensemble des salariés en tant que collaborateur, en conservant le rôle particulier des délégués et des responsables
- enregistrer de nouvelles informations essentielles dans chaque compte rendu (médicaments présentés, échantillons offerts, documentation remise)
- élaborer à terme des statistiques pour les délégués et les responsables
- envisager à terme que la nouvelle application puisse être visualisée sur divers types de terminaux (tablettes, smartphones)
- sécuriser l'application par un système d'authentification par profil

Il est essentiel de sécuriser l'application.

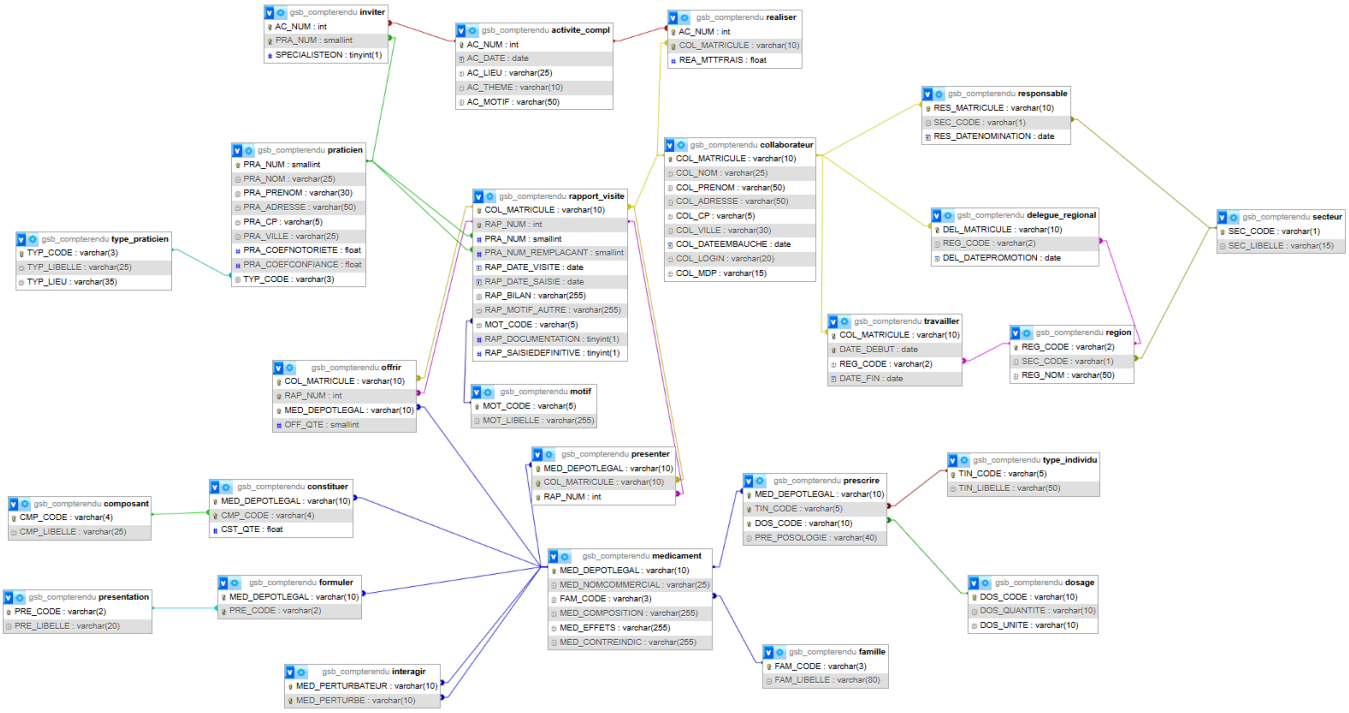
- Proposition de solution

Une application Web respectant l'architecture MVC sécurisée de gestion des comptes rendus des visiteurs médicaux (MySQL + PHP) peut être développée et intégrée à l'intranet de l'entreprise. Cette architecture garantit une séparation claire entre la logique métier, l'accès aux données et la présentation.

III. Analyse

A. Modélisation des données

Le schéma de données ci-dessous représente l'ensemble des entités gérées par l'application ainsi que leurs relations. La base de données gsb_compterendu contient 25 tables.

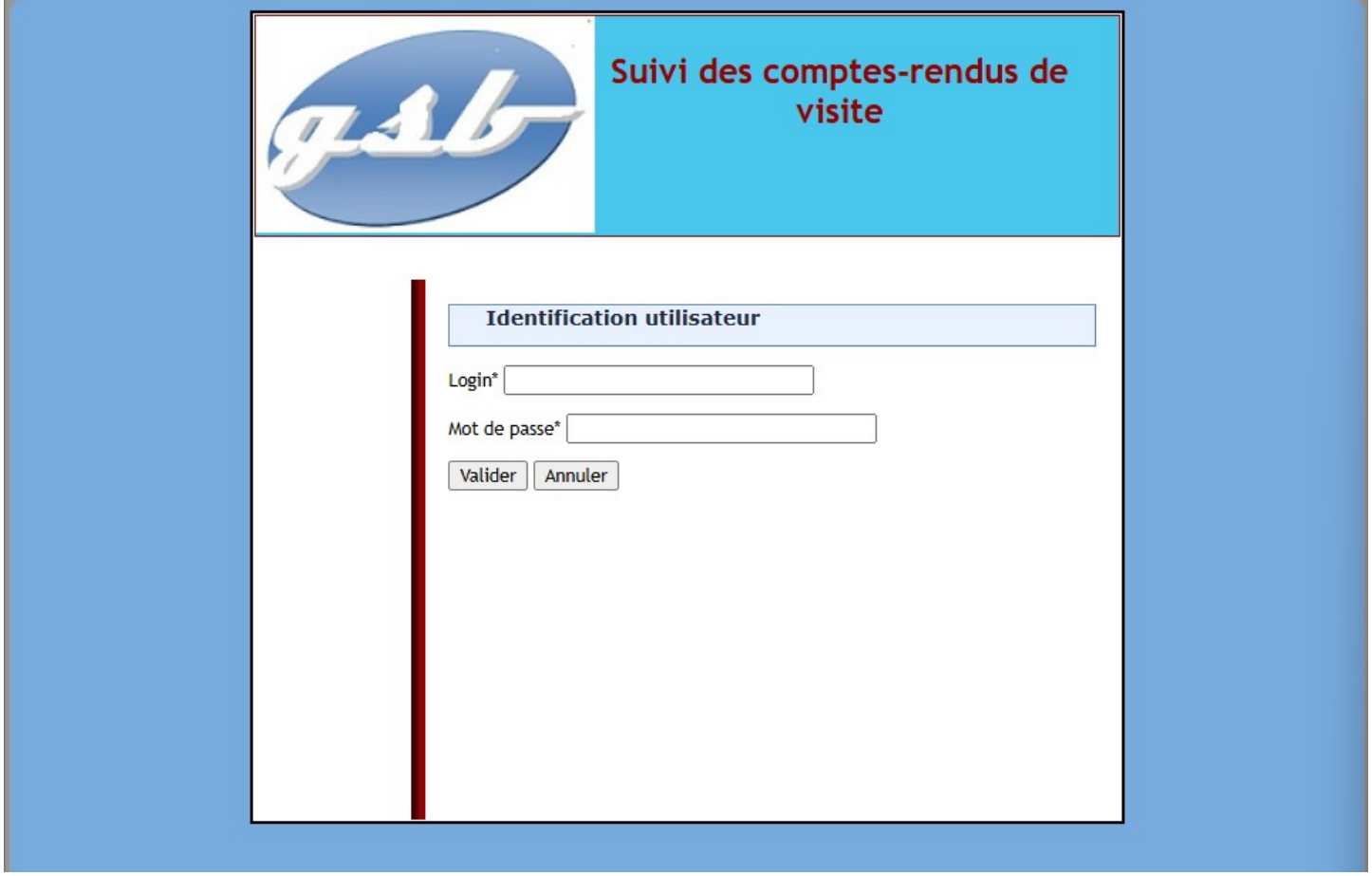


B. Les traitements

1. Connexion à l'application

La connexion est accessible à tous les profils (visiteur, délégué et responsable). Le système identifie le profil de l'utilisateur et l'oriente vers le menu correspondant à ses droits.

IHM — Interface visiteur, délégué et responsable

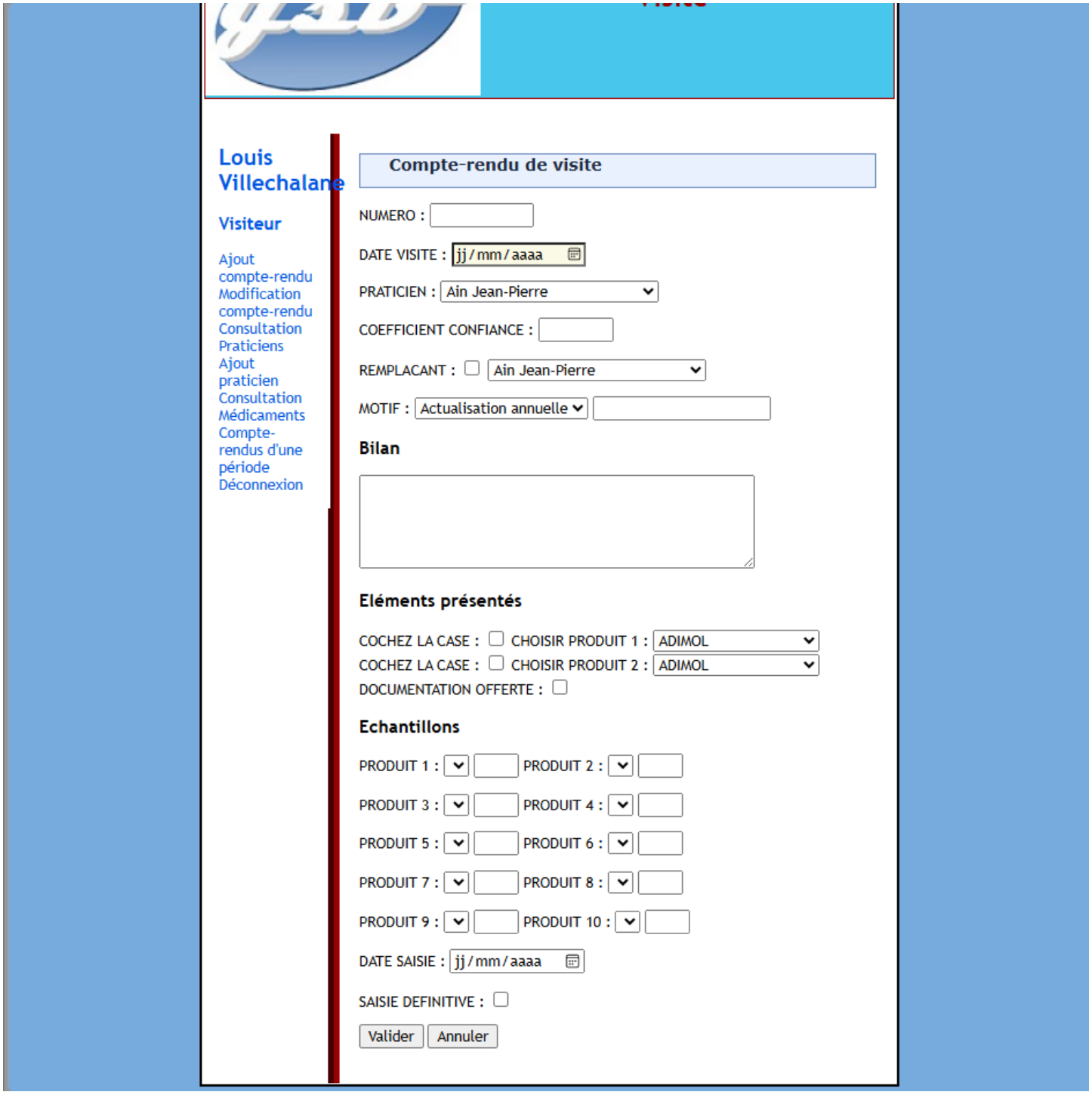


[CAPTURE D'ÉCRAN — Page de connexion de l'application (formulaire login / mot de passe)]

2. Saisie d'un compte rendu

Le visiteur ou le délégué peut saisir un nouveau compte rendu de visite médicale. Le formulaire inclut la date de visite, le praticien visité (et son éventuel remplaçant), le motif, le bilan libre, les médicaments présentés, les échantillons offerts et le statut de saisie.

IHM

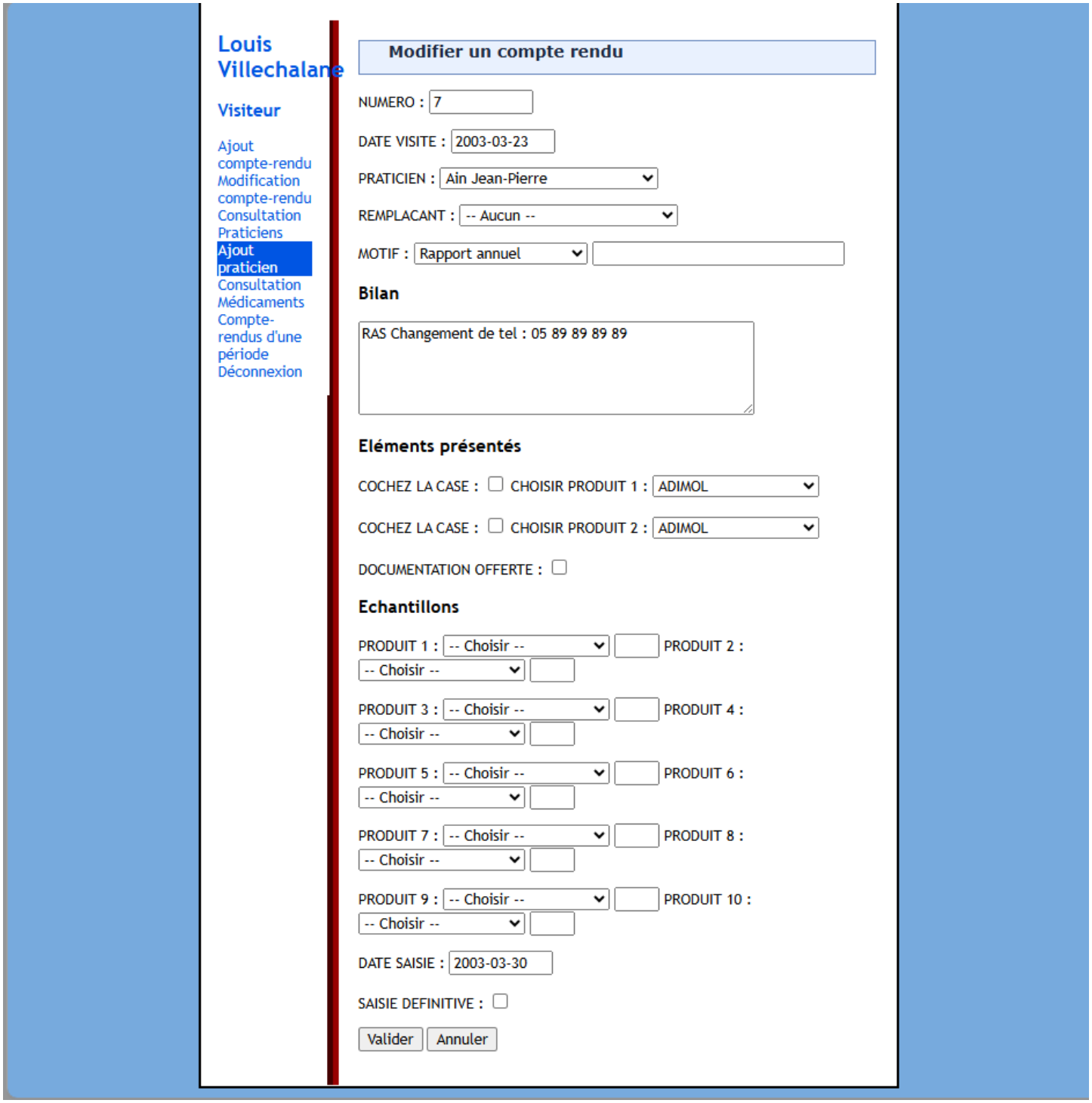


[CAPTURE D'ÉCRAN — Formulaire de saisie d'un compte rendu (v_ajouterCompteRendu.php)]

3. Modification d'un compte rendu

L'utilisateur sélectionne un compte rendu existant dans la liste de ses rapports, puis modifie les informations souhaitées. Seuls les comptes rendus non verrouillés (saisie non définitive) peuvent être modifiés.

IHM

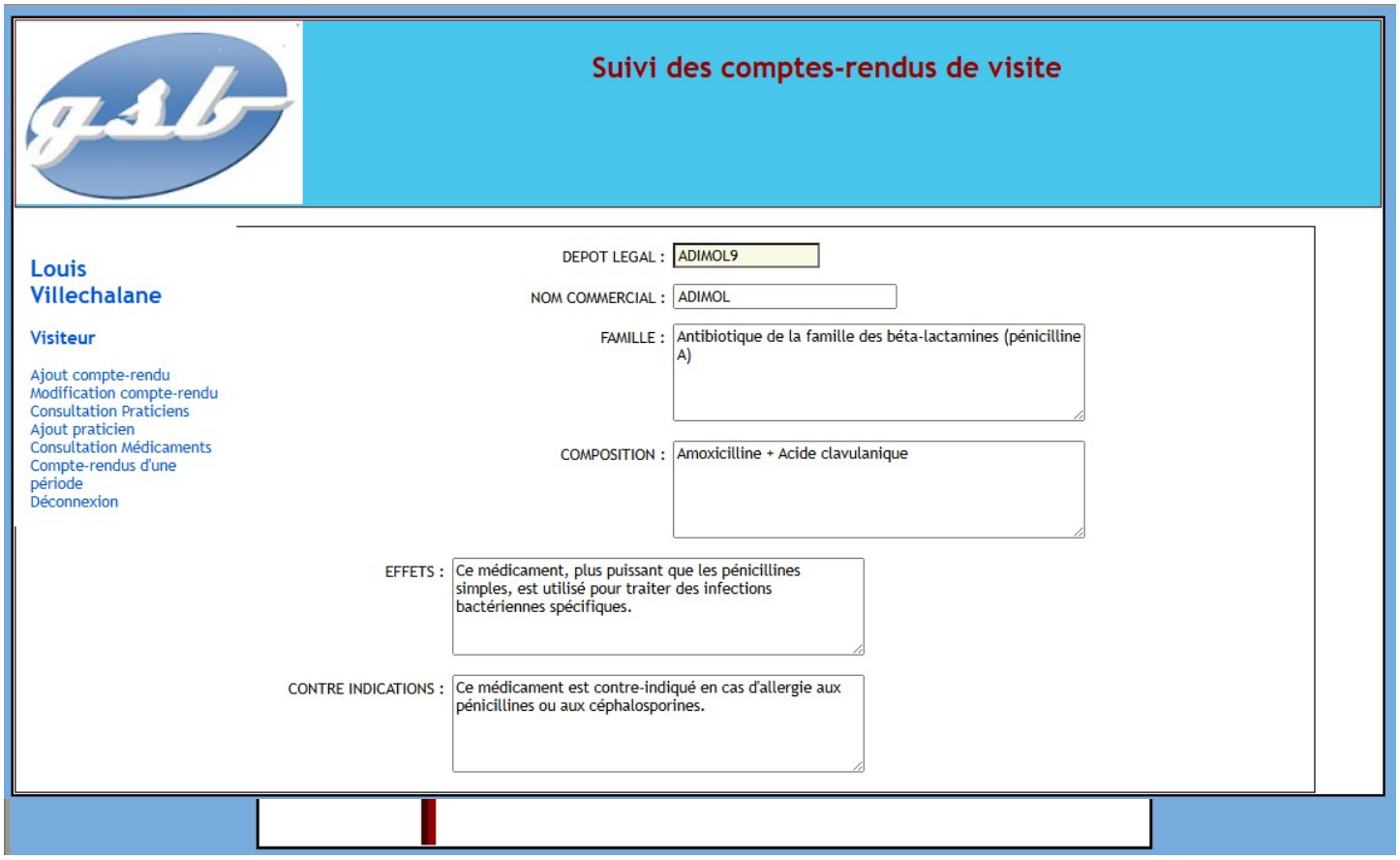


[CAPTURE D'ÉCRAN — Formulaire de sélection puis de modification d'un compte rendu (v_listeCompteRendu.php + v_majCompteRendu.php)]

4. Consultation des médicaments

Accessible aux visiteurs et aux délégués. L'utilisateur sélectionne un médicament dans la liste déroulante pour afficher sa fiche complète : dépôt légal, nom commercial, famille thérapeutique, composition, effets et contre-indications.

IHM

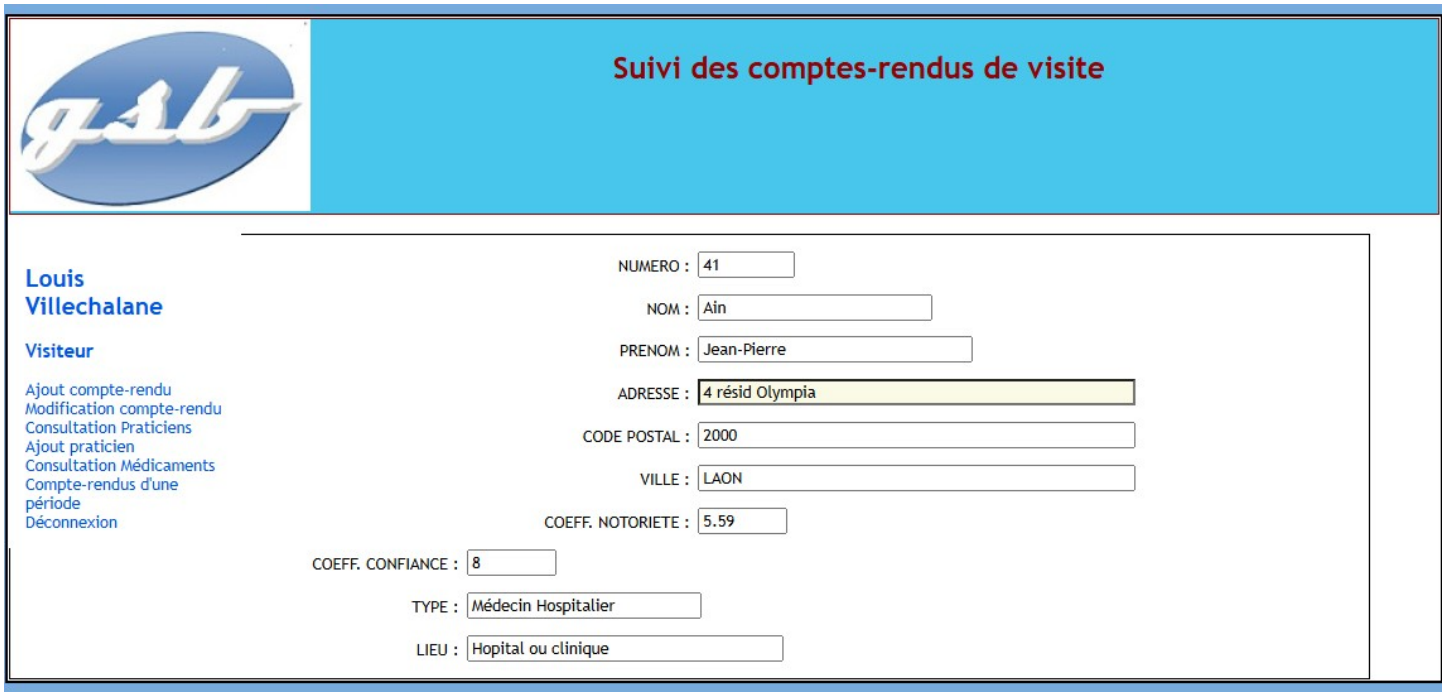


[CAPTURE D'ÉCRAN — Liste déroulante de sélection du médicament + fiche médicament (v_listeMedicament.php + v_consultMedicament.php)]

5. Consultation des praticiens

Accessible aux visiteurs et aux délégués. L'utilisateur sélectionne un praticien pour consulter sa fiche détaillée : coordonnées, coefficients de notoriété et de confiance, type et lieu d'exercice.

IHM



[CAPTURE D'ÉCRAN — Liste de sélection du praticien + fiche praticien (v_listePraticien.php + v_consultPraticien.php)]

6. Ajout d'un praticien

Accessible aux visiteurs et aux délégués. Le formulaire permet de créer un nouveau praticien en renseignant son numéro, son identité, son adresse, ses coefficients et son type.

IHM

[CAPTURE D'ÉCRAN — Formulaire d'ajout d'un praticien (v_ajoutPraticien.php)]

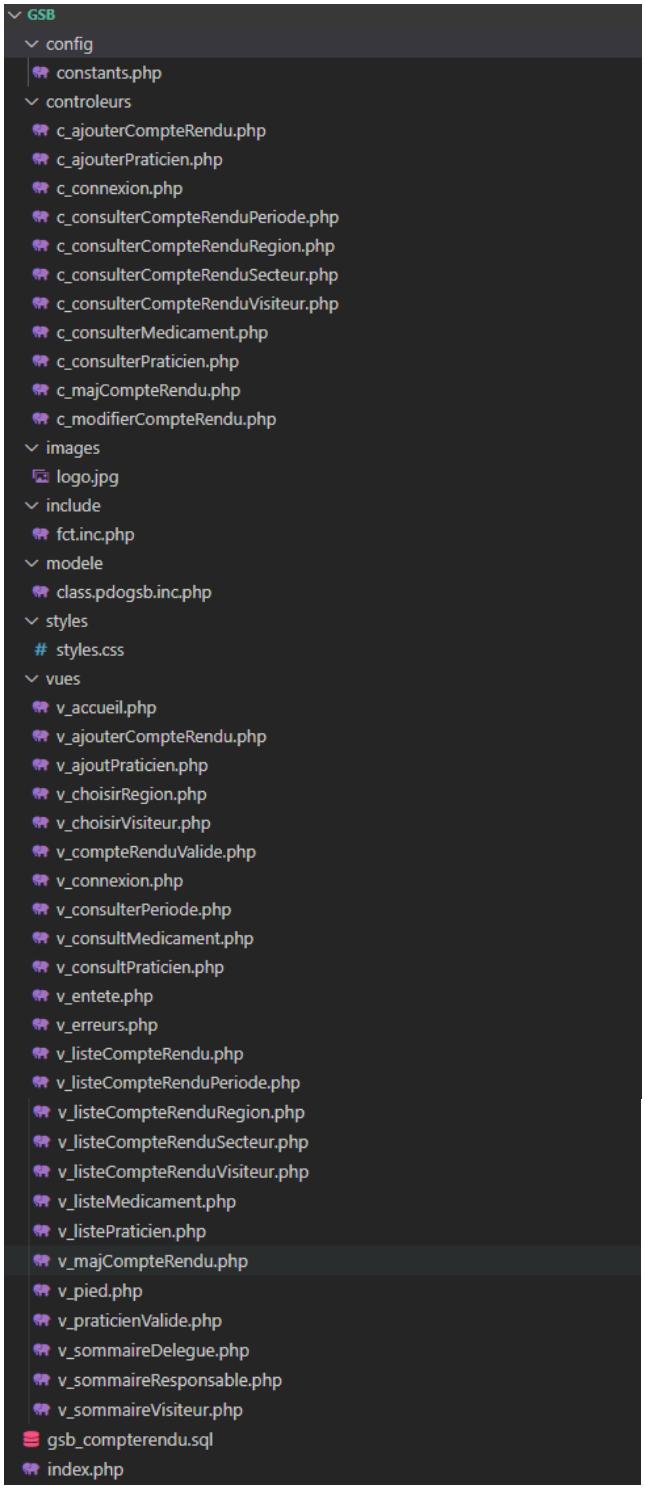
IV. Réalisation technique

A. Architecture de l’application

L’application respecte le patron de conception MVC (Modèle-Vue-Contrôleur). Le point d’entrée unique est le fichier index.php qui joue le rôle de contrôleur frontal. Les requêtes HTTP sont dirigées vers le bon contrôleur grâce aux paramètres uc (use case) et action.

Arborescence dossiers / fichiers

La structure du projet est la suivante :



[CAPTURE D’ÉCRAN — Arborescence complète du projet (dossiers controleurs/, vues/, modele/, include/, config/, styles/, images/)]

Répertoire / Fichier	Rôle
index.php	Point d'entrée unique — routeur frontal
config/constants.php	Constantes : paramètres BDD, chemins, URLs
include/fct.inc.php	Fonctions utilitaires (sessions, dates, erreurs)
modele/class.pdogsb.inc.php	Classe Singleton d'accès à la base MySQL
controleurs/c_*.php	Un contrôleur par cas d'utilisation (10 fichiers)
vues/v_*.php	Vues HTML/PHP — affichage des pages
styles/styles.css	Feuille de style CSS de l'application
images/logo.jpg	Logo du laboratoire Galaxy-Swiss Bourdin
gsb_compterendu.sql	Script SQL de création et peuplement de la base

B. L'accès à la base de données

Fichier : modele/class.pdogsb.inc.php

La classe PdoGsb centralise l'ensemble des accès à la base de données. Elle utilise le patron de conception Singleton : une seule instance de connexion PDO est créée et réutilisée pour toute la durée de la requête HTTP.

Principe du Singleton

Le constructeur est privé. La méthode statique getPdoGsb() retourne l'instance unique, en la créant si elle n'existe pas encore :

```
private static $monPdoGsb = null;

public static function getPdoGsb() {
    if (PdoGsb::$monPdoGsb == null) {
        PdoGsb::$monPdoGsb = new PdoGsb();
    }
    return PdoGsb::$monPdoGsb;
}
```

La connexion utilise PDO avec le jeu de caractères UTF-8 et le mode d'erreur par exceptions :

```
PdoGsb::$monPdo = new PDO(
    PdoGsb::$serveur . ';' . PdoGsb::$bdd,
    PdoGsb::$user,
    PdoGsb::$mdp
);
PdoGsb::$monPdo->query("SET CHARACTER SET utf8");
PdoGsb::$monPdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
```

Principe des fonctions (requêtes préparées)

Toutes les méthodes de la classe utilisent des requêtes préparées PDO (prepare / execute) pour garantir la sécurité contre les injections SQL. Les paramètres sont passés sous forme de tableau associatif :

```
public function getIdCollaborateur($login, $mdp) {
    $req = "SELECT COL_MATRICULE as matricule FROM COLLABORATEUR"
```

```
            . " WHERE COL_LOGIN = :login AND COL_MDP = :mdp";
    $stmt = PdoGsb::$monPdo->prepare($req);
    $stmt->execute(array(':login' => $login, ':mdp' => $mdp));
    return $stmt->fetch();
}
```

Le tableau ci-dessous récapitule l'ensemble des méthodes de la classe PdoGsb :

Méthode	Paramètres	Retour	Description
getIdCollaborateur()	$login, $mdp	tableau	Authentifie un collaborateur
estResponsable()	$matricule	booléen	Teste si le collaborateur est responsable
estDelegue()	$matricule	booléen	Teste si le collaborateur est délégué
getInfosCollaborateur()	$matricule	tableau	Retourne nom et prénom
getLesResponsables()	—	tableau[]	Liste tous les responsables
getLesDelegues()	—	tableau[]	Liste tous les délégués
getLesVisiteurs()	—	tableau[]	Liste les visiteurs (hors resp. et délég.)
getLesMedicaments()	—	tableau[]	Liste tous les médicaments
getUnMedicament()	$depotlegal	tableau	Fiche complète d'un médicament
getLesPraticiens()	—	tableau[]	Liste tous les praticiens
getUnPraticien()	$num	tableau	Fiche complète d'un praticien
getLesTypesPraticien()	—	tableau[]	Types de praticien
ajouterPraticien()	9 paramètres	void	Insère un nouveau praticien
getLesMotifs()	—	tableau[]	Liste les motifs de visite
getLesCompteRenduDuVisiteur()	$matricule	tableau[]	CR d'un visiteur (tri date desc)
getUnCompteRendu()	$numRapport	tableau	CR complet par numéro
ajouterCompteRendu()	11 paramètres	void	Insère un nouveau CR
modifierCompteRendu()	10 paramètres	void	Met à jour un CR existant
ajouterProduitPresente()	$depot, $matr, $num	void	Médicament présenté lors d'une visite
ajouterEchantillon()	$matr, $num, $depot, $qte	void	Échantillon offert lors d'une visite
getCompteRenduParPeriode()	$debut, $fin, $matr?	tableau[]	CR filtrés par période

getCompteRenduParRegion()	$codeRegion	tableau[]	CR d'une région
getCompteRenduParSecteur()	$codeSecteur	tableau[]	CR d'un secteur entier
getCompteRenduParVisiteur()	$matricule	tableau[]	CR d'un visiteur spécifique
getRegionDuDelegue()	$matricule	tableau	Région d'un délégué
getSecteurDuResponsable()	$matricule	tableau	Secteur d'un responsable
demarreTransaction()	—	void	beginTransaction()
valideTransaction()	—	void	commit()
annuleTransaction()	—	void	rollback()

C. Les fonctions utilitaires

Fichier : include/fct.inc.php

Ce fichier regroupe toutes les fonctions partagées entre les contrôleurs. Il est inclus dans chaque requête via index.php.

Gestion des sessions et de l'authentification

Fonction	Description
estConnecteVisiteur()	Retourne true si la session visiteur est ouverte
estConnecteDelegue()	Retourne true si la session délégué est ouverte
estConnecteResponsable()	Retourne true si la session responsable est ouverte
connecterVisiteur($m, $n, $p)	Initialise les variables de session pour un visiteur
connecterDelegue($m, $n, $p)	Initialise les variables de session pour un délégué
connecterResponsable($m, $n, $p)	Initialise les variables de session pour un responsable
deconnecter()	Détruit la session PHP (session_destroy())

Gestion des erreurs

Fonction	Description
ajouterErreur($msg)	Ajoute un message d'erreur au tableau $_REQUEST['erreurs']
nbErreurs()	Retourne le nombre d'erreurs enregistrées

Fonctions de validation et de conversion

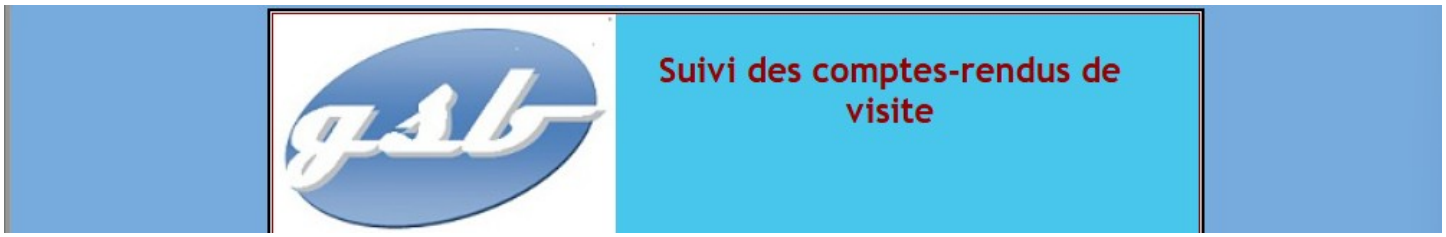
Fonction	Description
estEntierPositif($valeur)	Vérifie qu'une valeur est un entier positif ou nul
estTableauEntiers($tab)	Vérifie que tous les éléments d'un tableau sont

	entiers
estDateValide($date)	Valide le format jj/mm/aaaa et l'existence de la date
estDateDepassee($date)	Vérifie si une date est vieille de plus d'un an
dateFrancaisVersAnglais($date)	Convertit jj/mm/aaaa → aaaa-mm-jj (requêtes SQL)
dateAnglaisVersFrancais($date)	Convertit aaaa-mm-jj → jj/mm/aaaa (affichage)

D. Architecture d'une page

Chaque page de l'application est construite selon une structure commune définie dans index.php. Trois zones sont toujours présentes :

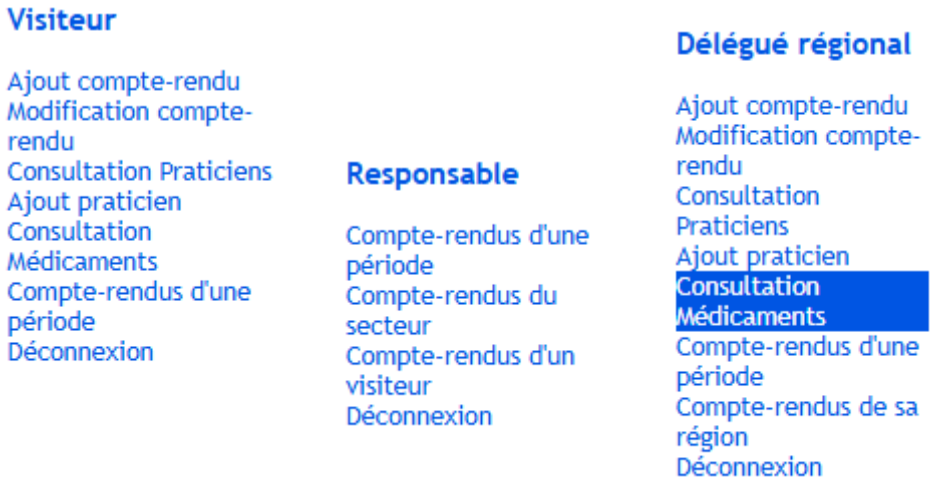
En-tête (v_entete.php)
Affiche le logo du laboratoire, le titre de l'application et importe la feuille de style CSS. Commun à toutes les pages.



[CAPTURE D'ÉCRAN — En-tête de l'application : logo GSB + titre "Suivi des comptes-rendus de visite"]

Sommaire (menu latéral gauche)
Affiché selon le profil de l'utilisateur connecté. Trois variantes existent :

- v_sommaireVisiteur.php — menu du visiteur médical
- v_sommaireDelegue.php — menu du délégué régional (avec accès à la consultation région)
- v_sommaireResponsable.php — menu du responsable de secteur



[CAPTURE D'ÉCRAN — Les trois menus latéraux : Visiteur / Délégué régional / Responsable, côte à côte]

Contenu variable (zone centrale)

La zone centrale est alimentée par la vue incluse par le contrôleur actif. Elle change à chaque action de l'utilisateur.

E. Les contrôleurs et les vues

index.php — Routeur frontal

index.php est le seul point d'entrée de l'application. Il démarre la session, charge la configuration, instancie PdoGsb et route vers le bon contrôleur via un switch sur le paramètre uc :

```
switch ($uc) {
    case 'connexion':
        include(CONTROLLERSPATH . 'c_connexion.php'); break;
    case 'ajouterCompteRendu':
        if ($estConnecteVisiteur || $estConnecteDelegue)
            include(CONTROLLERSPATH . 'c_ajouterCompteRendu.php');
        else include(VIEWSPATH . 'v_accessDenied.php');
        break;
    // ... (10 cas d'utilisation au total)
}
```

Un contrôleur par cas d'utilisation

Chaque contrôleur suit la même logique : il vérifie la connexion de l'utilisateur, lit le paramètre action, appelle les méthodes du modèle nécessaires, puis inclut la vue appropriée.

Contrôleur	Actions gérées	Accès
c_connexion.php	demandeConnexion, valideConnexion, deconnexion	Tous
c_ajouterCompteRendu.php	saisirCompteRendu, validerCompteRendu	Visiteur, Délégué
c_majCompteRendu.php	choisirCompteRendu, modifierCompteRendu, validerModification	Visiteur, Délégué
c_ajouterPraticien.php	saisirPraticien, validerPraticien	Visiteur, Délégué
c_consulterPraticien.php	choisirPraticien, voirPraticien	Visiteur, Délégué
c_consulterMedicament.php	choisirMedicament, voirMedicament	Visiteur, Délégué
c_consulterCompteRenduPeriode.php	choisirPeriode, voirCompteRenduPeriode	Tous
c_consulterCompteRenduRegion.php	choisirRegion, voirCompteRenduRegion	Délégué, Responsable
c_consulterCompteRenduSecteur.php	voirCompteRenduSecteur	Responsable
c_consulterCompteRenduVisiteur.php	choisirVisiteur, voirCompteRenduVisiteur	Responsable

Logique de traitement — exemple : c_connexion.php

Le contrôleur de connexion illustre la logique commune à tous les contrôleurs :

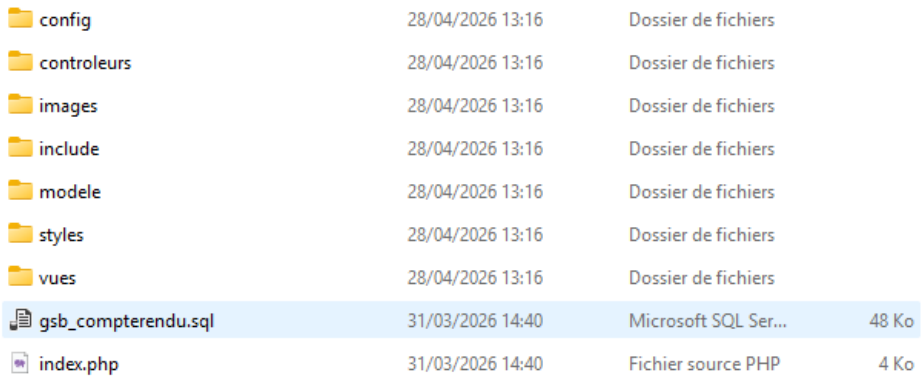
```
case 'valideConnexion': {
    $login = trim(htmlentities($_REQUEST['login']));
    $mdp   = trim(htmlentities($_REQUEST['mdp']));
    $collab = $pdo->getIdCollaborateur($login, $mdp);
    if (!is_array($collab)) {
        ajouterErreur("Login ou mot de passe incorrect");
        include(VIEWSPATH . 'v_erreurs.php');
        include(VIEWSPATH . 'v_connexion.php');
    } else {
        $matricule = $collab['matricule'];
        if ($pdo->estResponsable($matricule)) {
            $info = $pdo->getInfosResponsable($matricule);
            connecterResponsable($matricule, $info['nom'], $info['prenom']);
            include(VIEWSPATH . 'v_sommaireResponsable.php');
        } elseif ($pdo->estDelegue($matricule)) {
            $info = $pdo->getInfosDelegue($matricule);
            connecterDelegue($matricule, $info['nom'], $info['prenom']);
            include(VIEWSPATH . 'v_sommaireDelegue.php');
        } else {
            $info = $pdo->getInfosCollaborateur($matricule);
            connecterVisiteur($matricule, $info['nom'], $info['prenom']);
            include(VIEWSPATH . 'v_sommaireVisiteur.php');
        }
        include(VIEWSPATH . 'v_accueil.php');
    }
    break;
}
```

[CAPTURE D'ÉCRAN — Résultat de la connexion : page d'accueil affichée après authentification réussie (profil visiteur)]

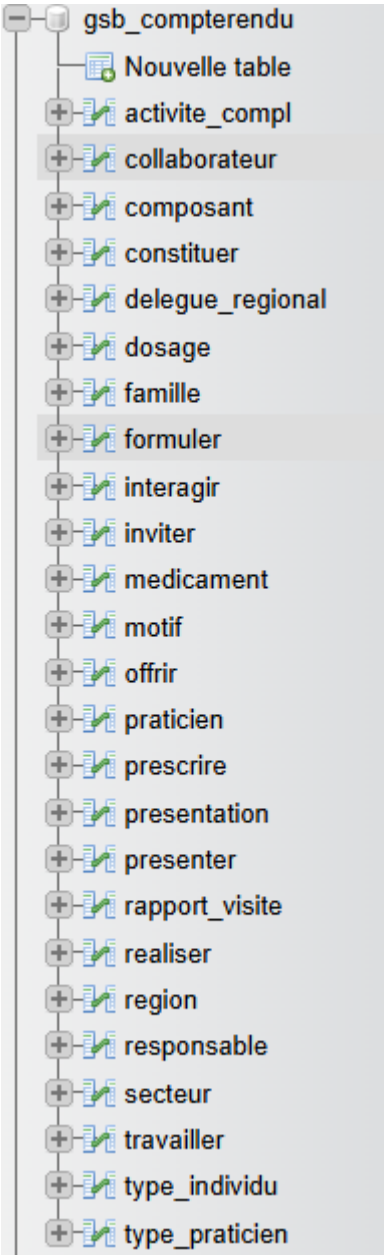
V. Procédure d'installation de l'application

A. Architecture physique

L'application est déployée sur un serveur qui héberge le serveur de base de données. Le reste est en Local sur Laragon..



config	28/04/2026 13:16	Dossier de fichiers	
controleurs	28/04/2026 13:16	Dossier de fichiers	
images	28/04/2026 13:16	Dossier de fichiers	
include	28/04/2026 13:16	Dossier de fichiers	
modele	28/04/2026 13:16	Dossier de fichiers	
styles	28/04/2026 13:16	Dossier de fichiers	
vues	28/04/2026 13:16	Dossier de fichiers	
gsb_compterendu.sql	31/03/2026 14:40	Microsoft SQL Ser...	48 Ko
index.php	31/03/2026 14:40	Fichier source PHP	4 Ko



Composant	Rôle	Emplacement
Serveur web Apache	Héberge et sert l'application PHP	Serveur
PHP 7.x+	Moteur de traitement des scripts	Serveur
MySQL 5.x+	Gestion de la base de données	Serveur
Extension PDO/PDO_MySQL	Connexion PHP vers MySQL	Serveur
Navigateur web	Interface utilisateur	Poste client

B. Étapes d'installation

- 1. Prérequis logiciels

- Serveur Apache avec module PHP activé (version 7.x ou supérieure recommandée)
- Serveur MySQL (version 5.x ou supérieure)

- Extensions PHP : PDO, PDO_MySQL
- Navigateur web récent (Firefox, Chrome, Edge)

- **2. Déploiement des fichiers**

Copier le dossier GSB_AAL-main dans le répertoire racine du serveur web :

```
# Exemple sous Windows (WAMP) :
C:\wamp64\www\GSB_AAL-main\

# Exemple sous Linux (Apache) :
/var/www/html/GSB_AAL-main/
```

- **3. Création de la base de données**

Importer le script SQL fourni avec l'application :

```
mysql -u root -p < gsb_compterendu.sql
```

Ou via l'interface phpMyAdmin : créer la base gsb_compterendu puis importer le fichier gsb_compterendu.sql.

- **4. Configuration de l'application**

Modifier le fichier config/constants.php pour adapter les paramètres à l'environnement :

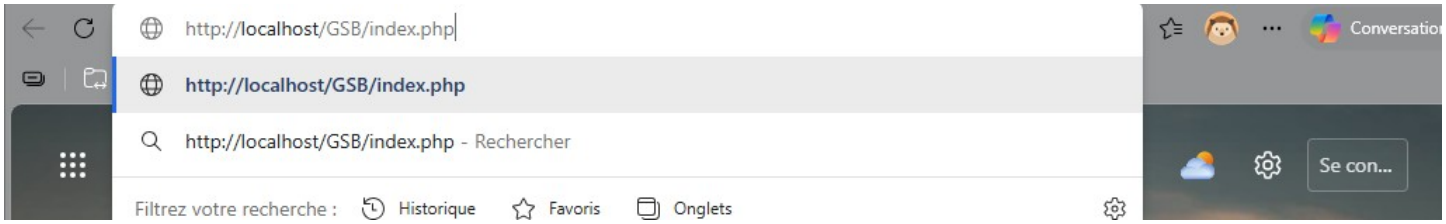
```
define('DB_HOST',  'mysql:host=localhost');
define('DB_NAME',  'dbname=gsb_compterendu');
define('DB_LOGIN', 'root');
define('DB_MDP',   '');
define('URL',      'http://localhost/GSB_AAL-main/');
```

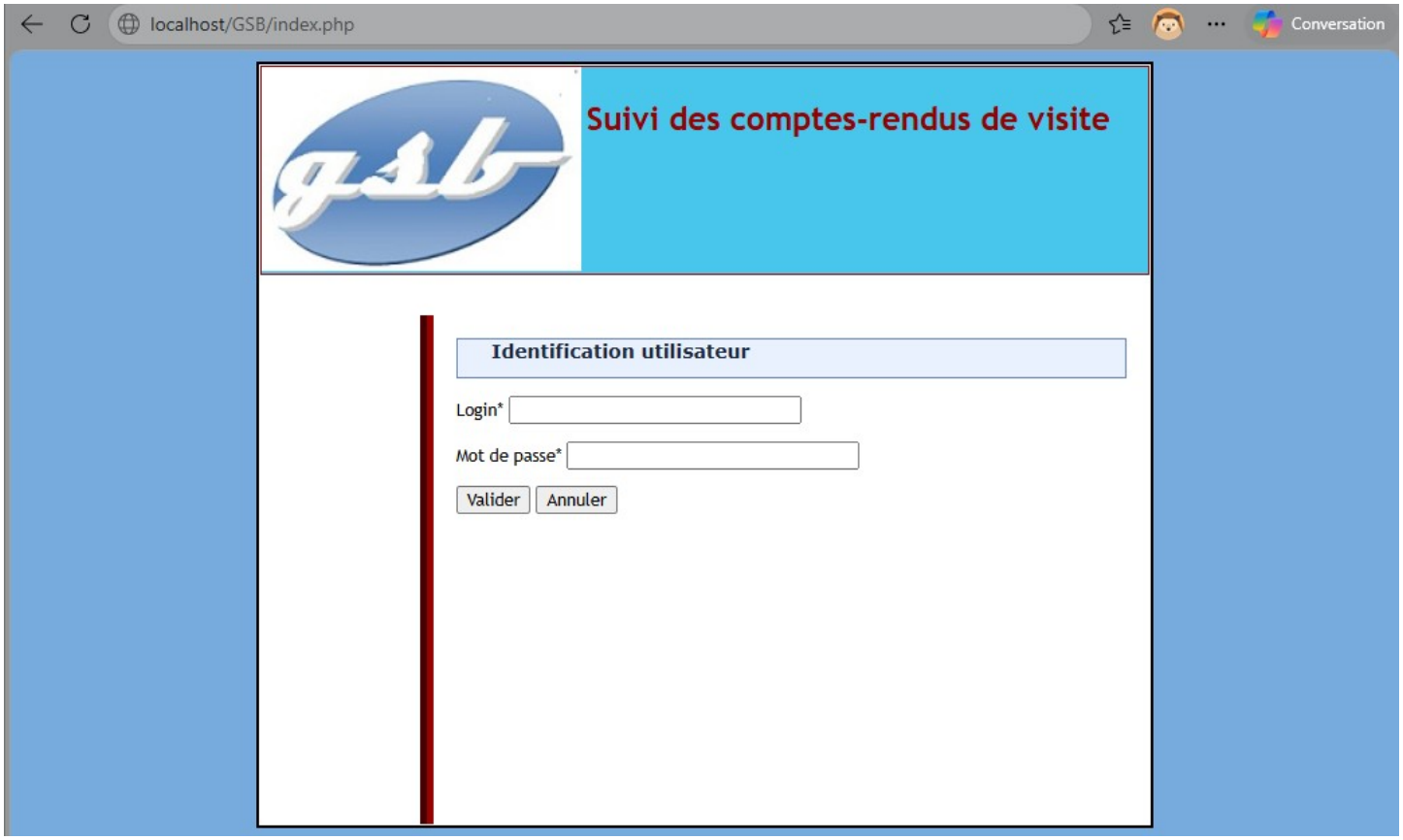
- **5. Vérification**

Accéder à l'application depuis un navigateur :

```
http://localhost/GSB/index.php
```

La page de connexion doit s'afficher. Se connecter avec un compte présent dans la base pour vérifier le bon fonctionnement de chaque profil.





[CAPTURE D'ÉCRAN — Page de connexion confirmant le bon déploiement de l'application]

— Fin du document —